Problem_Set2_code

October 24, 2026

0.1 Setup

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
```

0.2 Problem 1(a)

[2]:
```
wages = pd.read_csv("ps1small.csv")

wages['logwage'] = np.log(wages['wage'])
wages['eduxage'] = 'd'+ wages['education'].astype(str) + wages['age'].
 ↪astype(str)
dummies = pd.get_dummies(wages['eduxage'], dtype = int)
wages = pd.concat([wages, dummies], axis=1)
wages.head()
```

[2]:

	wage	education	age	logwage	eduxage	d1226	d1227	d1228	d1229	\
0	30000	17	30	10.308953	d1730	0	0	0	0	
1	16000	12	27	9.680344	d1227	0	1	0	0	
2	43500	17	28	10.680516	d1728	0	0	0	0	
3	18000	12	30	9.798127	d1230	0	0	0	0	
4	25000	17	27	10.126631	d1727	0	0	0	0	

	d1230	…	d1626	d1627	d1628	d1629	d1630	d1726	d1727	d1728	d1729	\
0	0	…	0	0	0	0	0	0	0	0	0	
1	0	…	0	0	0	0	0	0	0	0	0	
2	0	…	0	0	0	0	0	0	0	1	0	
3	1	…	0	0	0	0	0	0	0	0	0	
4	0	…	0	0	0	0	0	0	1	0	0	

	d1730
0	1
1	0
2	0
3	0
4	0

[5 rows x 35 columns]

[3]:
```
model = sm.OLS(endog= wages['logwage'], exog=wages[dummies.columns]).fit()
coefficients = model.params
standard_errors = model.bse
alpha = pd.DataFrame({'Coefficient': coefficients, 'Standard Error':␣
 ↪standard_errors})
alpha
```

[3]:

	Coefficient	Standard Error
d1226	9.451320	0.090780
d1227	9.300426	0.084980
d1228	9.537856	0.083280
d1229	9.385826	0.085421
d1230	9.628323	0.087737
d1326	9.612120	0.192010
d1327	9.755901	0.192010
d1328	9.378158	0.174517
d1329	9.819982	0.223685
d1330	9.446488	0.202991
d1426	9.580658	0.223685
d1427	9.794423	0.170843
d1428	9.642237	0.216101
d1429	9.712199	0.174517
d1430	9.645440	0.170843
d1526	9.534555	0.241608
d1527	9.385619	0.232129
d1528	9.573155	0.264668
d1529	9.540361	0.252351
d1530	9.919473	0.241608
d1626	10.070500	0.167391
d1627	9.944426	0.187149
d1628	9.991987	0.147954
d1629	10.178524	0.130710
d1630	10.070591	0.164140
d1726	9.958317	0.483216
d1727	10.045760	0.223685
d1728	10.165648	0.295908
d1729	9.617564	0.418477
d1730	10.015981	0.374297

alpha is the individual effect of having a given age and education on log wage.

0.3 Problem 1(b)

[4]:
```
R  = np.zeros((26,30))
q = np.zeros((26,1))

def col(e, a):
    # column index for education level e and age a
    return (e - 12) * 5 + (a - 26)

#alpha_{e,a-1} - 2*alpha_{e,a} + alpha_{e,a+1} = 0
r = 0
for e in range(12, 18):
    for a in (27, 28, 29):
        R[r, col(e, a-1)] = 1
        R[r, col(e, a)]   = -2
        R[r, col(e, a+1)] = 1
        r += 1

#(alpha_{e-1,27} - alpha_{e-1,26}) - 2*(alpha_{e,27} - alpha_{e,26}) +␣
 ↪(alpha_{e+1,27} - alpha_{e+1,26}) = 0
for r in range(18, 22):
    e = 13 + (r - 18)
    R[r, col(e-1, 27)] = 1
    R[r, col(e-1, 26)] = -1
    R[r, col(e, 27)]   = -2
    R[r, col(e, 26)]   = 2
    R[r, col(e+1, 27)] = 1
    R[r, col(e+1, 26)] = -1

#alpha_{e-1,26} - 2*alpha_{e,26} + alpha_{e+1,26} = 0
for r in range(22, 26):
    e = 13 + (r - 22)
    R[r, col(e-1, 26)] = 1
    R[r, col(e, 26)]   = -2
    R[r, col(e+1, 26)] = 1
```

[5]:
```
wald_test = model.wald_test((R,q), use_f = True)
F = wald_test.fvalue
p_value = wald_test.pvalue
print("F-statistic:", F)
print("p-value:", p_value)
```

```
F-statistic: 0.9199279217972832
p-value: 0.5805908886458206
```

I would not reject the null hypothesis as the p-value (0.58) is not below 0.05 significance level.

0.4 Problem 3(d)

[6]:
```
from scipy import stats
kappa = 5
# T ~ N(0, kappa-1) under the null, but the test uses the N(0,2) critical value␣
 ↪1.645*sqrt(2)
size = 1 - stats.norm.cdf(1.645*np.sqrt(2) / np.sqrt(kappa-1))
print(f'actual asymptotic size when kappa=5: {size:.4f}')
```

```
actual asymptotic size when kappa=5: 0.1224
```

0.5 Problem 3(e)

[7]:
```
from scipy import stats
```

[8]:
```
wage = pd.read_excel('wage.xlsx')
wage['logwage'] = np.log(wage['wage'])
wage.head()
```

[8]:

	male	wage	logwage
0	0	6.315296	1.842975
1	0	5.479770	1.701063
2	0	3.642170	1.292580
3	0	4.593337	1.524607
4	0	2.418158	0.883006

[9]:
```
wage_males = wage[wage['male']==1]
wage_females = wage[wage['male']==0]

n1 = len(wage_females)
n2 = len(wage_males)

y1 = wage_females['logwage']
y2 = wage_males['logwage']
y1_bar = np.mean(y1)
y2_bar = np.mean(y2)

s1 = np.sum((y1-y1_bar)**2) / (n1-1)
s2 = np.sum((y2-y2_bar)**2) / (n2-1)

T = np.sqrt(n1*n2/ (n1+n2)) * (np.log(s1) - np.log(s2))

k_hat = (n1+n2)* (np.sum((y1-y1_bar)**4) + np.sum((y2-y2_bar)**4)) / (np.
 ↪sum((y1-y1_bar)**2) + np.sum((y2-y2_bar)**2)) **2

z = T / np.sqrt(k_hat-1)
p_robust = 1- stats.norm.cdf(z)
```

```
F = s1/s2
p_normal = 1 - stats.f.cdf(F,n1-1,n2-1)

print(f'p-value robust to non-normality {p_robust}')
print(f'p value under normality {p_normal}')
```

```
p-value robust to non-normality 0.17481673039385381
p value under normality 0.04734658854081053
```

The p-value under normal assumptions shows that we would reject the null with 0.05 significance level, while the non-normality robust test showa we would not reject the null.